CNPM-65

LƯU VẾT ĐÁNH GIÁ MINH BẠCH

Báo cáo thiết kế và triển khai minh họa bằng Python

Jira: [11.2] — WBS 11.2 — SRS FR-11.2

Công nghệ: Python • Flask • SQLite

1. Mục tiêu

Tạo cơ chế lưu vết đánh giá của giám khảo nhằm đảm bảo tính minh bạch và khả năng truy xuất. Dữ liệu gồm giám khảo, tiêu chí, điểm, nhận xét, thời gian; sau khi xác nhận, bản ghi không thể chỉnh sửa hoặc xóa.

2. Chức năng triển khai

• Tạo bản ghi đánh giá ở trạng thái DRAFT.

• Lưu judge, submission ID, criterion, score, max score, comment và thời gian đánh giá.

• Cho phép sửa điểm/nhận xét khi bản ghi ở trạng thái nháp.

• Xác nhận bản ghi: trạng thái chuyển CONFIRMED và ghi thời điểm xác nhận.

• SQLite trigger chặn UPDATE và DELETE đối với bản ghi đã xác nhận.

• Audit trail ghi nhận tạo, sửa, xác nhận; có actor, timestamp, payload, hash SHA-256 và hash liên kết.

• Trang audit hỗ trợ tra cứu theo bài nộp, actor hoặc loại event.

3. Thiết kế dữ liệu

Bảng	Các trường chính
assessments	id, judge, submission_id, criterion, score, max_score, comment, assessed_at, status, confirmed_at, reco
audit_events	id, assessment_id, event, actor, event_time, payload, prev_hash, event_hash

4. Quy trình nghiệp vụ

Giám khảo tạo đánh giá → bản ghi lưu ở DRAFT → có thể chỉnh sửa nháp → giám khảo xác nhận → trạng thái CONFIRMED, hệ thống khóa UPDATE/DELETE → sự kiện được ghi vào audit trail.

5. Hướng dẫn cài đặt và chạy

Tạo virtual environment Python, cài thư viện bằng pip install -r requirements.txt, chạy python app.py và mở http://127.0.0.1:5000. SQLite database tự tạo khi khởi động.

6. Kịch bản kiểm thử

TC	Thao tác	Kết quả mong đợi
01	Tạo assessment với điểm hợp lệ	Tạo bản ghi DRAFT và event CREATED
02	Nhập điểm âm hoặc vượt max	Request bị từ chối HTTP 400
03	Sửa điểm/comment bản nháp	Dữ liệu cập nhật, event UPDATED được ghi
04	Xác nhận bản ghi	Status CONFIRMED, confirmed_at được lưu
05	Thử sửa bản ghi CONFIRMED	Bị chặn HTTP 403 / trigger
06	Thử xóa bản ghi CONFIRMED	SQLite trigger ngăn xóa
07	Mở trang /audit và tìm kiếm	Audit event được lọc, hiển thị hash chain
08	Gọi /health	HTTP 200, status=ok

7. Giới hạn và bảo mật

Demo hiện dùng judge identity trong session đơn giản, chưa tích hợp authentication thật. Cần bổ sung CSRF, RBAC, secret management, backup, audit database độc lập. Hash chain giúp phát hiện thay đổi trong chuỗi nhưng cần lưu/đóng dấu hash ở hệ thống ngoài để tăng khả năng chống sửa bởi người có quyền DB.

8. Mã nguồn Python (app.py)

```
from flask import Flask, request, render_template, redirect, url_for, session, abort, flash
import sqlite3, os, json, hashlib
from datetime import datetime, timezone

app = Flask(__name__)
app.secret_key = os.getenv("SECRET_KEY", "replace-with-a-long-secret")
DB_PATH = os.getenv("DB_PATH", "assessment_audit.db")

def db():
    c=sqlite3.connect(DB_PATH); c.row_factory=sqlite3.Row; return c
```

```
def init_db():
    with db() as c:
        c.executescript("""
        CREATE TABLE IF NOT EXISTS assessments (
          id INTEGER PRIMARY KEY AUTOINCREMENT,
          judge TEXT NOT NULL,
          submission_id TEXT NOT NULL,
          criterion TEXT NOT NULL,
          score REAL NOT NULL CHECK(score >= 0),
          max_score REAL NOT NULL CHECK(max_score > 0 AND score <= max_score),
          comment TEXT NOT NULL DEFAULT '',
          assessed_at TEXT NOT NULL,
          status TEXT NOT NULL CHECK(status IN ('DRAFT','CONFIRMED')) DEFAULT 'DRAFT',
          confirmed_at TEXT,
          record_hash TEXT
        );
        CREATE TABLE IF NOT EXISTS audit_events (
          id INTEGER PRIMARY KEY AUTOINCREMENT,
          assessment_id INTEGER NOT NULL,
          event TEXT NOT NULL,
          actor TEXT NOT NULL,
          event_time TEXT NOT NULL,
          payload TEXT NOT NULL,
          prev_hash TEXT NOT NULL DEFAULT '',
          event_hash TEXT NOT NULL,
          FOREIGN KEY(assessment_id) REFERENCES assessments(id)
        );
        """)
        # SQLite triggers enforce immutability after confirmation.
        c.executescript("""
        CREATE TRIGGER IF NOT EXISTS prevent_confirmed_update
        BEFORE UPDATE ON assessments
        WHEN OLD.status = 'CONFIRMED'
        BEGIN SELECT RAISE(ABORT, 'Confirmed assessment is immutable'); END;
        CREATE TRIGGER IF NOT EXISTS prevent_confirmed_delete
        BEFORE DELETE ON assessments
        WHEN OLD.status = 'CONFIRMED'
        BEGIN SELECT RAISE(ABORT, 'Confirmed assessment cannot be deleted'); END;
        """)

def utcnow():
    return datetime.now(timezone.utc).astimezone().isoformat(timespec="seconds")

def write_event(assessment_id, event, actor, payload):
    with db() as c:
        last=c.execute("SELECT event_hash FROM audit_events ORDER BY id DESC LIMIT 1").fetchone()
        prev=last["event_hash"] if last else ""
        when=utcnow()
        body=json.dumps({"assessment_id":assessment_id,"event":event,"actor":actor,
                         "event_time":when,"payload":payload,"prev_hash":prev},
                        sort_keys=True,ensure_ascii=False)
        digest=hashlib.sha256(body.encode("utf-8")).hexdigest()
        c.execute("INSERT INTO audit_events(assessment_id,event,actor,event_time,payload,prev_hash,event_hash) VALUES(?,?,?,?,?,?,?)",
                  (assessment_id,event,actor,when,json.dumps(payload,ensure_ascii=False),prev,digest))

@app.route("/")
def index():
    actor=session.get("judge","judge_demo")
    with db() as c:
        rows=c.execute("SELECT * FROM assessments ORDER BY id DESC LIMIT 100").fetchall()
    return render_template("index.html",rows=rows,actor=actor)

@app.route("/set-judge",methods=["POST"])
def set_judge():
    judge=(request.form.get("judge") or "judge_demo").strip()[:100]
    session["judge"]=judge
    return redirect(url_for("index"))

@app.route("/assessment/create",methods=["POST"])
def create_assessment():
    judge=session.get("judge","judge_demo")
    submission=(request.form.get("submission_id") or "").strip()
    criterion=(request.form.get("criterion") or "").strip()
    comment=(request.form.get("comment") or "").strip()
    try: score=float(request.form.get("score",""))
    except ValueError: return "Điểm phải là số.",400
    try: max_score=float(request.form.get("max_score",""))
    except ValueError: return "Điểm tối đa phải là số.",400
    if not submission or not criterion or max_score<=0 or score<0 or score>max_score:
        return "Dữ liệu không hợp lệ: cần submission, tiêu chí và điểm trong khoảng 0..điểm tối đa.",400
    assessed=utcnow()
    with db() as c:
        cur=c.execute("INSERT INTO assessments(judge,submission_id,criterion,score,max_score,comment,assessed_at,status) VALUES(?,?,?,?,?,?,?,'DRAFT')",
                      (judge,submission,criterion,score,max_score,comment,assessed))
        aid=cur.lastrowid
    write_event(aid,"ASSESSMENT_CREATED",judge,{"score":score,"max_score":max_score,"criterion":criterion})
    return redirect(url_for("index"))

@app.route("/assessment/<int:aid>/confirm",methods=["POST"])
def confirm(aid):
    actor=session.get("judge","judge_demo")
    with db() as c:
        row=c.execute("SELECT * FROM assessments WHERE id=?",(aid,)).fetchone()
        if not row: abort(404)
        if row["status"]=="CONFIRMED": return "Bản ghi đã xác nhận; không thể sửa hoặc xác nhận lại.",409
        confirmed=utcnow()
        c.execute("UPDATE assessments SET status='CONFIRMED',confirmed_at=? WHERE id=?",(confirmed,aid))
    write_event(aid,"ASSESSMENT_CONFIRMED",actor,{"confirmed_at":confirmed})
    return redirect(url_for("index"))

@app.route("/assessment/<int:aid>/edit",methods=["POST"])
def edit(aid):
    actor=session.get("judge","judge_demo")
    with db() as c:
        row=c.execute("SELECT * FROM assessments WHERE id=?",(aid,)).fetchone()
        if not row: abort(404)
        if row["status"]=="CONFIRMED": abort(403,"Bản ghi đã xác nhận — không được chỉnh sửa.")
        try: score=float(request.form.get("score",""))
        except ValueError: return "Điểm không hợp lệ.",400
        comment=(request.form.get("comment") or "").strip()
        if score<0 or score>row["max_score"]: return "Điểm ngoài giới hạn.",400
```

```
        c.execute("UPDATE assessments SET score=?,comment=?,assessed_at=? WHERE id=?",
                  (score,comment,utcnow(),aid))
    write_event(aid,"ASSESSMENT_UPDATED",actor,{"score":score,"comment":comment})
    return redirect(url_for("index"))

@app.route("/audit")
def audit():
    q=(request.args.get("q") or "").strip()
    with db() as c:
        rows=c.execute("""SELECT e.*,a.submission_id,a.criterion,a.status
                          FROM audit_events e JOIN assessments a ON a.id=e.assessment_id
                          WHERE (?='' OR a.submission_id LIKE ? OR e.actor LIKE ? OR e.event LIKE ?)
                          ORDER BY e.id DESC LIMIT 300""",
                       (q,f"%{q}%",f"%{q}%",f"%{q}%")).fetchall()
    return render_template("audit.html",rows=rows,q=q)

@app.route("/health")
def health(): return {"status":"ok","service":"assessment-audit"},200

if __name__=="__main__":
    init_db()
    app.run(host="127.0.0.1",port=5000,debug=False)
```